

Implement API Endpoints Using Express-Mongo



Estimated time: 90 minutes

Congratulations! You are one step closer to finishing the capstone project. You created a web application and added user authentication using the Django framework and REACT in the previous modules. Take a pause and pat yourself on the back. However, it is not done yet!

The Django application will talk to MongoDB using backend services. You are asked to create these services in JavaScript. You will write these services in an Express app and deploy it on Code Engine.

Working with Mongoose to Provide API Endpoints

- Open a **New Terminal** and clone your project from GitHub if the environment has been reset.
- Change to the directory with the data files.



A terminal window with a light gray header bar containing a code icon, a purple 'bash' button, and a copy icon. The main area has a light blue background and shows the command `cd /home/project/xrwvm-fullstack_developer_capstone/s`. A green 'Run' button is on the right.

For your online course lab project, you have been provided two schema files for Reviews and Dealerships entities, along with JSON files containing dealership and review data to be loaded into MongoDB and served through endpoints.

- `server/database/data/dealerships.json`
- `server/database/data/reviews.json`
- The Node app will use `mongoose` to interact with the MongoDB. The schemas for Reviews and Dealerships are defined in `review.js` and `dealership.js`, respectively.
- View the content in `server/database/app.js`. It will provide the endpoints as shown below:
- `fetchReviews` (for fetching all reviews)
- `fetchReviews/dealer/:id` (for fetching reviews of a particular dealer)
- `fetchDealers` (for fetching all dealerships)
- `fetchDealers/:state` (for fetching all dealerships in a particular state)

- `fetchDealer/:id` (for dealer by id)
- `insert_review` (for inserting reviews)

Some of the endpoints have been implemented for you. Use the ideas and prior learning to implement the endpoints that are not implemented.

- Run the command given below to build the Docker app.

Remember to do this, every time you make changes to `app.js`.

<>

bash

```
docker build . -t nodeapp
```

Run

The first time you build, it takes up to two minutes to successfully build.

- The `docker-compose.yml` has been created to run two containers, one for Mongo and the other for the Node app. Run the command given below to run the server:

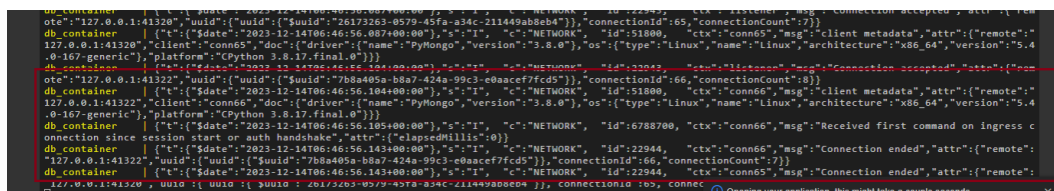
<>

bash

```
docker-compose up
```

Run

- When you see the output on the terminal as shown in the following image, it would mean that the server has successfully started, and you can connect to it.



- Select **Fetch Reviews** below to test if the API endpoint returns all the reviews as intended.

Fetch Reviews

► If the page doesn't open, select here to open the URL manually.

Create API Endpoint URLs

- Implement the endpoints given below that are yet to be implemented in `server/database/app.js`.
 - `fetchDealers`
 - `fetchDealers/:state`
 - `fetchDealer/:id`
- Stop your Docker application that you started in the previous task.
- Execute the `docker build` and `docker compose` commands again.
- Select **Fetch Reviews** below to test all endpoints by replacing the route in the address bar.

Fetch Reviews

► If the page doesn't open, select here to open the URL manually.

Assessment:

For Option 1: AI-Graded Submission and Evaluation:

Test the endpoints listed below using this command format in your terminal:



plaintext



```
curl -X GET "<Backend-URL>/<endpoint>"
```

Note: Replace the Backend URL and the correct endpoint.

After running each command, copy the full terminal output (including the command itself) and save it in a text file with the required name.

- `/fetchReviews/dealer/` - Save it in a text file named `getdealerreviews`.
- `/fetchDealers` - Save it in a text file named `getalldealers`.
- `/fetchDealer/` - Save it in a text file named `getdealerbyid`.
- `/fetchDealers/Kansas` - Save it in a text file named `getdealersbyState`.

For Option 2: Peer-Graded Submission and Evaluation:

Test the following endpoints in the browser, take a screenshot of the same and save as specified.

- `/fetchReviews/dealer/29` - Save the screenshot as `dealer_review.png` or `dealer_review.jpeg`.
- `/fetchDealers` - Save the screenshot as `dealerships.png` or `dealerships.jpeg`.
- `/fetchDealer/3` - Save the screenshot as `dealer_details.png` or `dealer_details.jpeg`.
- `/fetchDealers/Kansas` - Save the screenshot as `kansasDealers.png` or `kansasDealers.jpeg`.
- Push the updated `app.js` code to your GitHub.

Summary

In this lab, you created a containerized Node.js application that uses MongoDB as a backend to serve API endpoints.

Author(s)

Lavanya

Other Contributor(s)

Upkar Lidder Yan Luo

© Copyright IBM Corporation. All rights reserved.

Summary

In this lab, you created a containerized Node.js application that uses MongoDB as a backend to serve API endpoints.

Author(s)

Lavanya

Other Contributor(s)

Upkar Lidder Yan Luo

← Create API Endpoint URLs